Computing Matroid Tilings of the Hypersimplex $\Delta(3, n)$: Algorithms, C++ Implementation, and Experimental Results

Juan Luis Vargas Molina

Draft manuscript, April 28, 2026

Abstract

This paper documents a C++ implementation for enumerating and validating rank–3 matroid tilings of the hypersimplex $\Delta(3, n)$, with special emphasis on the computational transition from $\Delta(3, 8)$ to $\Delta(3, 9)$. The implementation uses rank–3 matroids encoded by essential flat inequalities, represents matroid base polytopes by bit masks of hypersimplex vertices, tests face compatibility by a facet-closure criterion, and searches for face-fitting volume-complete tilings by an exact state-cache depth-first enumeration. For $n \leq 8$ the cell vertex sets fit in a single machine word; for $n = 9$ a separate wide-mask path uses a two-limb 128-bit representation. We report the validation strategy, the benchmark-driven tuning process, and the principal mathematical, algorithmic, and micro-architectural optimizations. The current $\Delta(3, 8)$ engine is a high-throughput quotient enumerator whose best measured profiles on a dual Threadripper-class system use static-hybrid scheduling, hot compatibility rows, and worker-local compatibility caches. The $\Delta(3, 9)$ engine is an experimental wide engine with exact final $\mathfrak{S}_9$ quotient deduplication but labeled partial-state caching; its observed low instructions per cycle indicates that the main remaining bottlenecks are memory hierarchy, data layout, NUMA locality, and dense compatibility-row traffic. The paper is both a computational report and a roadmap: it identifies the implemented ideas, the limitations of the present search, and the optimizations needed to make $\Delta(3, 9)$ computations substantially more efficient.

1 Introduction

Hypersimplices and matroid polytopes form a bridge between combinatorics, tropical geometry, and compactifications of moduli spaces. The rank–r hypersimplex is the polytope

$$\Delta(r, n) = \operatorname{conv}\{e_{i_1} + \cdots + e_{i_r} : \{i_1, \ldots, i_r\} \subset [n]\} \subset \mathbb{R}^n.$$

Its vertices are naturally indexed by r-subsets of $[n]$. A rank–r matroid M on $[n]$ determines a matroid base polytope

$$P(M) = \operatorname{conv}\{e_B : B \in \mathcal{B}(M)\} \subseteq \Delta(r, n).$$

The computational problem studied here is to enumerate face-fitting collections of rank–3 matroid base polytopes whose union covers $\Delta(3, n)$ and whose volumes add to the volume of the ambient hypersimplex.

The motivation comes from Valery Alexeev's theory of weighted stable hyperplane arrangements. Alexeev's Barcelona notes describe weighted stable hyperplane arrangements as one of

the few higher-dimensional stable-pair settings where concrete computations are possible, and emphasize that the subject involves an interplay among the Minimal Model Program, GIT, matroid theory, and polytopal tilings. Chapter 4 of those notes is devoted to matroid polytopes and tilings, and Chapter 5 applies this combinatorics to weighted stable hyperplane arrangements. In dimension two this corresponds to arrangements of lines in projective planes, hence to rank–3 matroid geometry.

This paper is a formal report on the implementation effort. It is not a claim that the full $\Delta(3,8)$ or $\Delta(3,9)$ classification has been completed in the present manuscript. Rather, it documents the computational model, the correctness checks, the data formats, the measured effects of several optimization families, and the remaining bottlenecks. This distinction is important: the implementation can generate and validate tilings, and it has been validated against known small cases, but exhaustive counts for larger n should be reported only after a complete run with reproducible logs.

Contributions

The main contributions documented here are the following.

1. A rank–3 C++ model for matroid cells in $\Delta(3,n)$ using essential inequalities $x(S) \leq k$ and vertex masks of 3-subsets.
2. An exact face-fitting test based on facet-closure in the vertex set, avoiding general polyhedral computations in the inner loop.
3. A high-throughput quotient search for $n \leq 8$ using labeled cell images, by-vertex membership bitsets, lazy compatibility rows, forced-cell propagation, and exact canonical final deduplication.
4. A benchmark-driven performance-tuning workflow for $\Delta(3,8)$, including static-hybrid scheduling, hot compatibility-row profiling, worker-local compatibility caches, binary state keys, and auto-profile selection.
5. A separate $\Delta(3,9)$ wide-mask path based on a two-limb `Mask128` representation, designed not to disturb the optimized $n \leq 8$ path.
6. A clear diagnosis of why the $n = 9$ case is qualitatively harder: 84 hypersimplex vertices, tens of millions of labeled images in the current dataset, enormous image-index bitsets, and low observed IPC on Threadripper-class hardware.
7. A reproducibility and profiling framework: JSONL inputs, deterministic text output, DOT diagrams, Python-generated plots, benchmark matrices, and scripts for rebuilding figures.

2 Mathematical background

Definition 1 (Hypersimplex)**.** *For $0 \leq r \leq n$, the hypersimplex $\Delta(r,n)$ is the convex hull of the $\{0,1\}$ vectors in $\mathbb{R}^n$ with coordinate sum r:*

$$\Delta(r,n) = \left\{ x \in [0,1]^n : \sum_{i=1}^{n} x_i = r \right\}.$$

For this project $r = 3$, and vertices of $\Delta(3,n)$ are indexed by triples $B \subset [n]$.

Definition 2 (Matroid by bases)**.** *A rank–3 matroid M on $[n]$ is represented by its set of bases $\mathcal{B}(M) \subseteq \binom{[n]}{3}$ satisfying the basis exchange axiom. The implementation assumes that the input orbit representatives have already been generated and validated as rank–3 connected loopless matroids.*

Definition 3 (Matroid base polytope)**.** *The base polytope of a rank–3 matroid M is*

$$P(M) = \operatorname{conv}\{e_B : B \in \mathcal{B}(M)\} \subseteq \Delta(3, n).$$

The program represents $P(M)$ by the bitset of vertices B appearing as bases.

Definition 4 (Matroid tiling)**.** *A matroid tiling of $\Delta(3, n)$ is a finite collection $\{P(M_i)\}$ of matroid base polytopes such that the cells are pairwise face-fitting, their union covers $\Delta(3, n)$, and the sum of their volumes equals the volume of the hypersimplex in the selected volume convention.*

Convention 1 (Volume)**.** *The default volume used by the implementation is the "sha-volume" supplied by the input files. For the full hypersimplex $\Delta(3, n)$ this volume is $(n-3)^2$. A lattice-volume field is also stored and can be used as a cross-check.*

2.1 Rank–3 inequalities

For rank 3, the input files encode each cell by a list of essential inequalities

$$x(S) \le k, \qquad k \in \{1, 2\}.$$

The always-present constraints $0 \le x_i \le 1$ and $\sum_i x_i = 3$ are not listed. The vertex set of the cell is therefore

$$V(M) = \left\{ B \in \binom{[n]}{3} : |B \cap S| \le k \text{ for every listed inequality } x(S) \le k \right\}.$$

This is the central specialization that makes the implementation fast: all cell geometry is reduced to operations on subsets of $\binom{[n]}{3}$.

3 Stable weighted hyperplane arrangements

Weighted stable hyperplane arrangements provide compactifications of moduli spaces of log canonical hyperplane arrangements

$$\left(\mathbb{P}^{r-1}, \sum_{i=1}^{n} b_i B_i \right).$$

Alexeev's program reduces concrete questions about degenerations to stable toric and matroid-polytopal data. In rank 3, the corresponding projective configurations are arrangements of lines in $\mathbb{P}^2$, and the relevant matroids record incidences such as coincident lines and multiple intersection points. The tilings of $\Delta(3, n)$ are combinatorial shadows of degenerations of the corresponding arrangements.

The implementation deliberately works at the level of abstract matroids. It does not restrict to realizable matroids, and it does not use oriented matroids. This is not a bug: the combinatorial compactification framework naturally involves matroid polytopes, and restricting to realizable or oriented data would change the enumerative problem.

Figure 1 summarizes the conceptual chain.

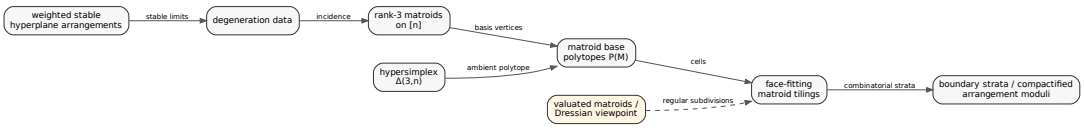

Figure 1: Mathematical correspondence used by the implementation. Rank–3 matroids encode incidence data, matroid base polytopes are cells inside $\Delta(3, n)$, and face-fitting matroid tilings encode combinatorial degeneration data. Dashed arrows indicate the additional regular-subdivision viewpoint through valuated matroids and the Dressian.

4 Computational problem

The program takes as input an integer n and a JSONL file of rank–3 connected loopless matroid orbit representatives. Each orbit representative contains:

$$n, \quad \texttt{orbit\_id}, \quad \texttt{ineqs}, \quad \texttt{volume\_sha}, \quad \texttt{volume\_lattice}.$$

For $n \leq 8$, the supplied data include $1, 4, 15, 52, 187$ orbit representatives for $n = 4, 5, 6, 7, 8$ respectively. The $n = 8$ input has 187 orbit representatives and the uniform orbit has `volume_sha`$= 25$. In the current $n = 9$ experiments the wide engine used a dataset with 901 orbit representatives and $70,731,920$ labeled cell images.

Computational fact 1 (Scale)**.** *The working scales are shown in Table 1. The transition from $n = 8$ to $n = 9$ changes both the cell-mask type and the number of labeled images by two orders of magnitude.*

Table 1: Scale of the implemented rank–3 computations. The $n = 9$ image count is from the current wide-engine benchmark input and should be regarded as dataset-dependent.

n	$\binom{n}{3}$	orbit reps	labeled images	sha-volume	mask type
4	4	1	1	1	`uint64_t`
5	10	4	36	4	`uint64_t`
6	20	15	801	9	`uint64_t`
7	35	52	19,638	16	`uint64_t`
8	56	187	788,763	25	`uint64_t`
9	84	901	70,731,920	36	`Mask128`

The output is a stream of tilings, in either text or JSONL form. The canonical text format is

```
Delta(3,n) | k | [cell] + [cell] + ....
```

Cell labels are printed as comma-separated inequalities, with 1-based element labels.

5 Representation choices

The implementation separates the mathematical objects from their performance-oriented representations.

5.1 Hypersimplex vertices

The vertices of $\Delta(3, n)$ are all triples $B \subset [n]$. For $n = 8$ there are 56 vertices, so a cell vertex set fits in one `uint64_t`. For $n = 9$ there are 84 vertices, so a two-limb representation is used.

Listing 1: The wide mask type used by the $\Delta(3, 9)$ path.

```
struct alignas(16) Mask128 {
    uint64_t lo;
    uint64_t hi;

    bool empty() const noexcept { return (lo | hi) == 0; }
    unsigned popcount() const noexcept {
        return std::popcount(lo) + std::popcount(hi);
    }
    Mask128 operator&(Mask128 other) const noexcept {
        return {lo & other.lo, hi & other.hi};
    }
    Mask128 operator|(Mask128 other) const noexcept {
        return {lo | other.lo, hi | other.hi};
    }
    bool subset_of(Mask128 other) const noexcept {
        return ((lo & ~other.lo) | (hi & ~other.hi)) == 0;
    }
};
```

The $n = 9$ path intentionally does not replace the $n \leq 8$ path. This preserves the measured-fast $\Delta(3, 8)$ implementation and allows $n = 9$ to be optimized independently.

5.2 Image-index bitsets

The set of labeled cell images is much larger than the set of hypersimplex vertices. The implementation therefore uses two levels of bitsets:

cell mask over hypersimplex vertices, image-index bitset over labeled cell images.

For $n = 8$, an image-index dense row has about 98.6KB. For $n = 9$, with $70,731,920$ images, a dense image-index row has about 8.4MB. This single fact explains why the $n = 9$ path is dominated by memory hierarchy rather than arithmetic throughput.

5.3 Structure of arrays

The hot image metadata are stored as parallel arrays:

`vmask[]`, `volume[]`, `orbit[]`, `perm[]`, `vertex_count[]`.

Search and compatibility tests mostly touch vertex masks and volumes; output formatting touches orbit and permutation data much later. This layout reduces cache pollution relative to an array-of-structures representation.

6 Algorithms

6.1 Cell construction

Given a hypersimplex object and a list of inequalities, construct the cell vertex mask as follows.

Listing 2: Cell vertex construction.

```
Input: inequalities (S_a,k_a), precomputed le_mask[S][k].
mask := all_vertices
for each inequality (S,k):
    mask := mask AND le_mask[S][k]
return mask
```

The correctness invariant is immediate from the definition: a vertex B remains in the mask exactly when it satisfies every inequality.

6.2 Face-fitting by facet closure

For two cells P_i, P_j, define

$$I = V(P_i) \cap V(P_j).$$

If $I = \emptyset$ or $|I| = 1$, then I is automatically a face of both cells. Otherwise I is tested by intersecting all coordinate and essential-equality facets that contain it. For a cell P and a candidate intersection I, the test is:

Listing 3: Facet-closure face test.

```
closure := V(P)
for every coordinate equality E:
    if I subset E:
        closure := closure AND E
for every essential inequality x(S)<=k of P:
    E := vertices satisfying x(S)=k
    if I subset E:
        closure := closure AND E
return closure == I
```

Proposition 1. *The facet-closure test is exact for the cells used by the implementation, provided the listed coordinate and essential matroid inequalities define the relevant cell facets.*

Implementation-level proof sketch. A face of a polytope defined by linear inequalities is obtained by turning a subset of valid inequalities into equalities. The algorithm intersects exactly those listed equality vertex sets that contain the candidate intersection. If their intersection recovers the candidate, the candidate is a face. Conversely, if the candidate is a face supported by the available facet description, the supporting equalities all contain it and the closure recovers it. □

6.3 Compatibility cache

Compatibility rows are computed lazily. The key observation is the two-hit filter: only cells sharing at least two hypersimplex vertices need an actual face test. Cells sharing zero or one vertex are compatible.

Listing 4: Two-hit compatibility row construction.

```
seen := empty image bitset
two  := empty image bitset
for vertex v in V(P_i):
    two  := two OR (seen AND by_vertex[v])
    seen := seen OR by_vertex[v]

compat := all images
for j in set bits of two:
    inter := V(P_i) AND V(P_j)
    if popcount(inter) > 1 and not face_of_both(i,j,inter):
        compat.reset(j)
return compat
```

6.4 Search with state cache

The production $n \leq 8$ engine uses exact quotient state caching. A recursive state contains:

$$(\text{chosen cells},\ \text{covered vertex mask},\ \text{volume sum},\ \text{allowed images}).$$

The invariant is that every allowed image is compatible with every chosen cell.

Listing 5: Production DFS schema for $n \leq 8$.

```
search(state):
    if volume_sum > total_volume: return
    apply forced-cell propagation
    if volume_sum == total_volume:
        accept iff covered == all_vertices
        canonicalize final tiling under S_n
        emit if new
        return

    v := uncovered vertex with fewest feasible candidate images
    candidates := allowed images covering v and volume <= remaining
    order candidates by new coverage, rarity, volume, and deterministic id
    for cell in candidates:
        child := state + cell
        child.allowed := state.allowed AND compat(cell)
        if quotient partial key is new:
            search(child)
```

For $n = 9$, the current wide path uses labeled partial-state caching and exact final $\mathfrak{S}_9$ quotient deduplication. This is conservative: it may repeat isomorphic partial work, but it does not undercount final tiling orbits.

7 Implementation architecture

The implementation is modular. The core components are:

Component	Role
`HypersimplexR3`	triples, coordinate masks, inequality masks for $n \leq 8$
`HypersimplexR3Wide`	wide-mask version for $n = 9$
`CellOrbit`	orbit representative read from JSONL
`ImageSet`	distinct labeled images under $\mathfrak{S}_n$
`CompatCache`	lazy compatibility rows, hot-row recording
`SearchEngine`	state-cache DFS and solution streaming
`CanonicalLabeler`	quotient keys under ground-set relabeling
`BenchmarkRunner`	gather-data matrix and JSON performance records

Figure 2 shows the production data flow.

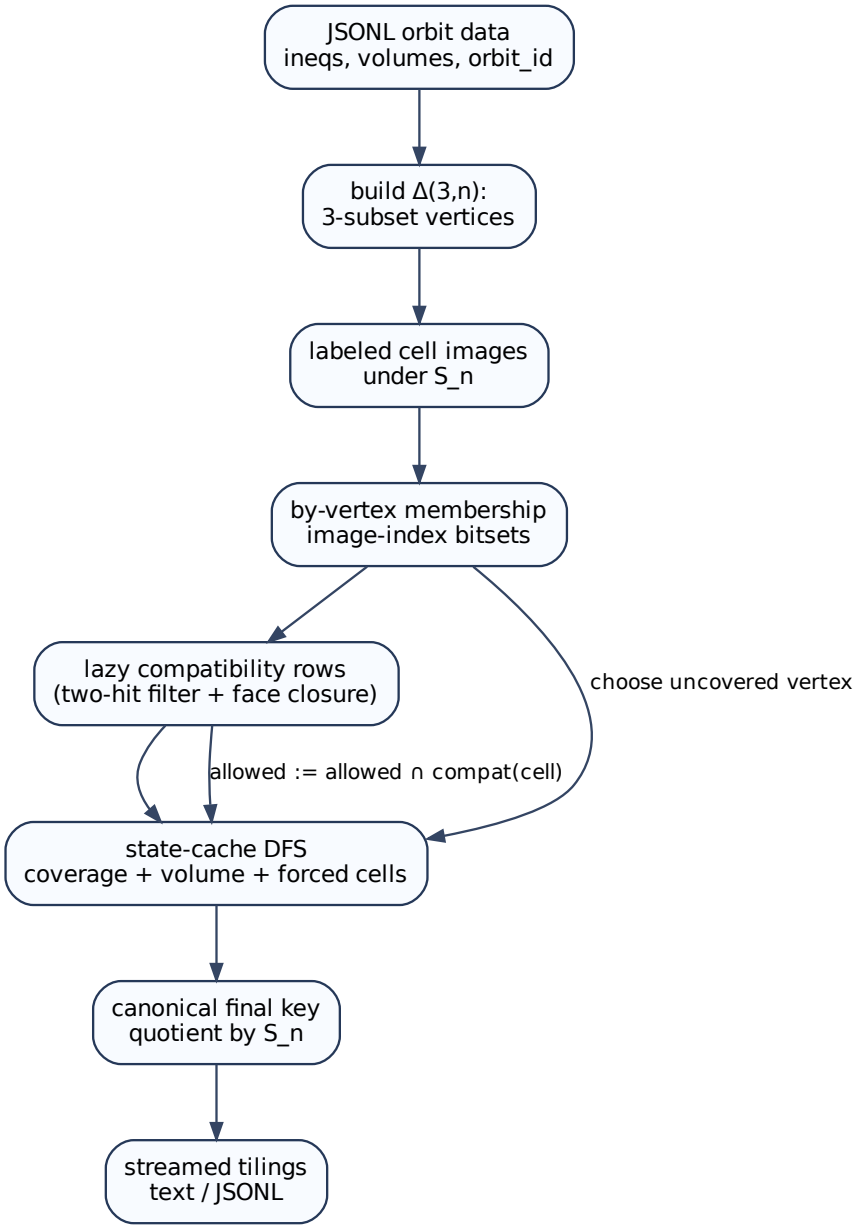

Figure 2: Implementation pipeline. The same mathematical pipeline is used for $n \leq 8$ and $n = 9$, but the underlying mask and cache types differ.

8 Optimizations implemented

The optimization history is important because several plausible improvements were measured and rejected. The implementation follows a benchmark-gated policy: changes are promoted into `--auto` only when they improve the measured production path.

8.1 Mathematical optimizations

1. **Rank–3 inequality encoding.** A cell is defined by essential inequalities, and vertices are triples satisfying simple cardinality tests.
2. **Facet-closure face tests.** No general convex-hull or face-lattice engine is used in the hot path.
3. **Volume pruning.** Candidate cells with volume larger than the remaining volume are ignored.
4. **Forced-cell propagation.** If an uncovered hypersimplex vertex can be covered by only one feasible compatible cell, that cell is forced.
5. **Quotient deduplication.** For $n \leq 8$, partial and final states are canonicalized under $\mathfrak{S}_n$; for $n = 9$, final states are quotient-deduplicated exactly.

8.2 Algorithmic optimizations

1. **Labeled-image generation.** Each orbit representative is expanded under the symmetric group, with duplicate vertex masks removed.
2. **By-vertex membership.** For each hypersimplex vertex, a bitset stores all cell images containing it.
3. **Two-hit compatibility filtering.** Only cells sharing at least two vertices are face-tested.
4. **Lazy compatibility rows.** Rows are computed on demand and cached.
5. **Hot-row profiling.** The program records frequently used compatibility rows and prewarms top rows in later runs.
6. **Static-hybrid scheduling.** The fast traversal order is preserved, but late subtree donation improves utilization.
7. **Worker-local compatibility cache.** A small per-worker cache avoids repeated global compatibility lookups.

8.3 Implementation and micro-architecture optimizations

1. **Single-word cell masks for $n \leq 8$.** Cell vertex sets are `uint64_t`.
2. **Wide `Mask128` for $n = 9$.** The $n = 9$ path uses two limbs and avoids changing the $n = 8$ engine.
3. **Binary state keys.** Canonical keys are stored as binary bytes rather than printable hex.

4. **Structure-of-arrays metadata.** Hot fields are separated from output-only fields.
5. **Cache-line padding.** Several shared counters and cache headers are aligned to reduce false sharing.
6. **CPU-affinity experiments.** NUMA affinity is available, though not always beneficial for $n = 8$ throughput.
7. **PGO/LTO/native build support.** The build system exposes flags for profile-guided and link-time optimization.

Figure 3 summarizes the optimized $n = 8$ architecture.

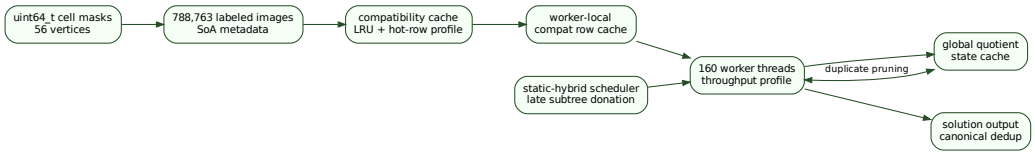

Figure 3: High-throughput $\Delta(3, 8)$ architecture. The measured-fast path uses `uint64_t` cell masks, static-hybrid scheduling, exact quotient state caching, hot compatibility rows, and worker-local compatibility caches.

9 Experimental results for $\Delta(3, 8)$

The $n = 8$ benchmark target used throughout the tuning process was time to first 10,000 emitted tilings. This is not a final enumeration count. It is a reproducible proxy for throughput, traversal quality, and time-to-output.

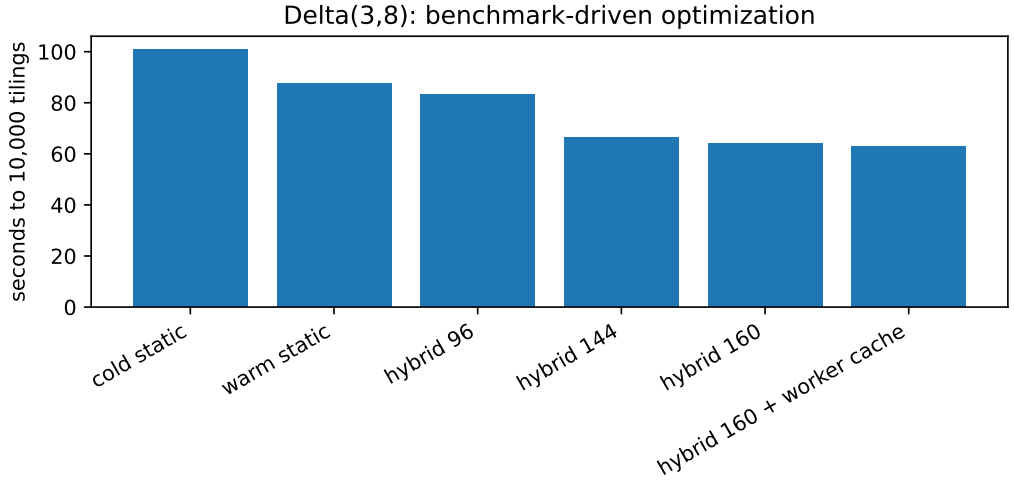

Figure 4: Representative $\Delta(3, 8)$ benchmark progression. Each bar is seconds to first 10,000 emitted tilings on a dual Threadripper-class system. The strongest profiles use static-hybrid scheduling and worker-local compatibility caching.

Computational fact 2. *In the benchmark logs, the best measured* $n = 8$ *throughput profiles are around* 63 *seconds to the first* 10,000 *tilings on the test machine, using approximately* 160 *threads, static-hybrid scheduling, LRU compatibility caching, and a worker-local compatibility cache of size* 64.

Several attempted changes were rejected as production defaults:

- full work stealing, because it changed traversal order and reached early tilings more slowly;
- static depth–2 splitting, because it produced a severe time-to-output regression;
- thread-local state cache, because global duplicate pruning is essential;
- labeled-state prefiltering, because it added overhead without sufficient pruning;
- forcing unrolled or AVX bitset kernels, because the measured auto kernel was better in the available data.

10 The $\Delta(3, 9)$ wide path

The transition from $n = 8$ to $n = 9$ is not merely a larger instance. It changes the basic machine representation. The 84 vertices of $\Delta(3, 9)$ do not fit in a `uint64_t`; moreover, the current $n = 9$ dataset used in benchmarking expands to 70,731,920 labeled images. A dense image-index row therefore has roughly

$$\left\lceil \frac{70{,}731{,}920}{8} \right\rceil \approx 8.4 \text{ MiB}$$

of payload. A computation that touches such rows frequently is not limited by integer arithmetic; it is limited by memory bandwidth, cache locality, TLB behavior, NUMA placement, and synchronization.

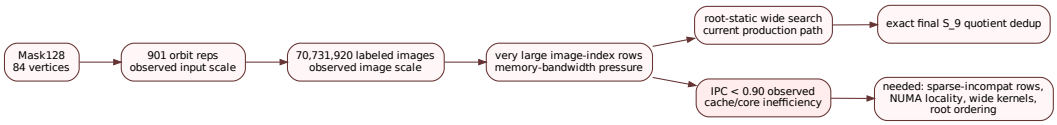

Figure 5: The isolated $\Delta(3, 9)$ wide path. The wide engine uses `Mask128` cell masks and exact final $\mathfrak{S}_9$ quotient deduplication, but the present partial-state cache is labeled. The principal bottleneck is dense image-index row traffic.

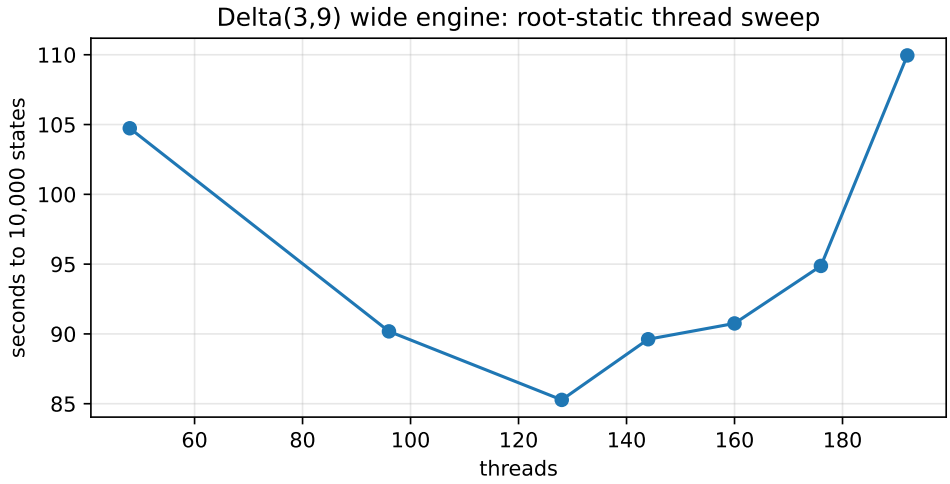

Figure 6: $\Delta(3, 9)$ root-static thread sweep from a state-bounded benchmark. The best observed point in the initial data is near 128 threads; higher thread counts become slower, indicating contention or memory-bandwidth saturation.

10.1 Observed IPC problem

On Threadripper-class systems the $n = 9$ path has shown instructions-per-cycle below 0.90 in profiling. This indicates a severe mismatch between the data structures and the hardware. Low IPC is consistent with the following pathologies:

- scanning dense multi-megabyte bitsets for rows that may be mostly compatible;
- cross-NUMA memory traffic from shared caches and shared bitsets;
- cache-line and TLB pressure from large image-index rows;
- lock and queue contention in global caches;
- poor temporal locality from root tasks that touch disjoint row sets;
- two-limb mask operations interleaved with large memory scans.

Figure 7 summarizes the diagnosis.

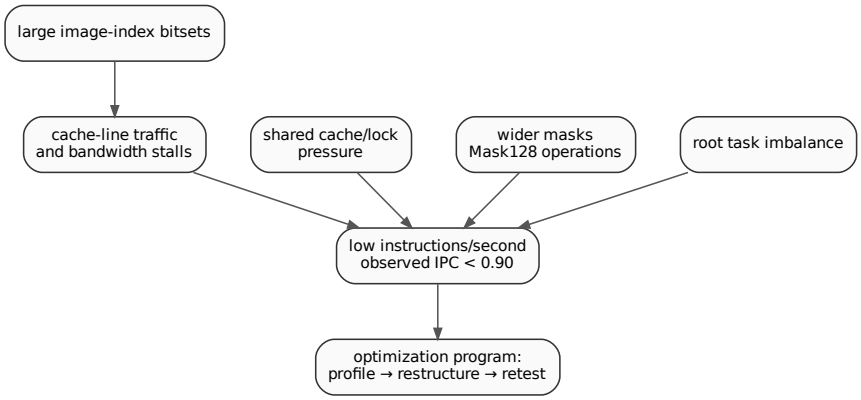

Figure 7: Performance interpretation for $\Delta(3, 9)$. The low IPC observation points toward memory stalls and layout problems rather than insufficient arithmetic instructions. The next optimizations should therefore target sparse row representations, NUMA locality, and bitset-kernel specialization.

11 Optimizations still needed for $\Delta(3, 9)$

The following optimizations are the most important next steps for the $n = 9$ case.

11.1 Sparse-incompatible compatibility rows

For $n = 8$, dense compatibility rows are small enough to be practical. For $n = 9$, dense rows are multi-megabyte objects. If most images are compatible with a given image, a row should be stored as

$$\text{all images} \setminus \{\text{incompatible images}\}.$$

The operation

$$\texttt{allowed} := \texttt{allowed} \cap \texttt{compat}(i)$$

then becomes a copy of `allowed` followed by clearing a sparse list of incompatible indices. This is exact and could reduce memory bandwidth by orders of magnitude for rows with few incompatibilities.

11.2 Hierarchical and compressed image-index sets

A flat dense bitset over 70 million images is too coarse. Possible alternatives include:

- block-sparse bitsets with a dense block only when needed;
- Roaring-style compressed bitmaps;
- volume- and orbit-partitioned candidate sets;
- two-level bitsets: active blocks plus in-block masks.

These representations should be benchmarked against actual compatibility-row density.

11.3 NUMA-local process or cache sharding

The $n = 9$ dataset is large enough that single-process shared structures may be counterproductive. A promising direction is:

$$\text{many processes} \times \text{moderate threads per process},$$

with disjoint root or prefix tasks, local compatibility caches, and final external canonical-key merge. This avoids cross-socket cache-line bouncing and improves first-touch locality.

11.4 Quotient partial-state canonicalization for $n = 9$

The current $n = 9$ partial-state cache is labeled. This is safe but repeats isomorphic work. A future exact quotient cache should avoid brute $\mathfrak{S}_9$ scans at every node. Candidate approaches include faithful colored-graph canonicalization, canonical root-prefix augmentation, or a verified refinement key that never merges non-isomorphic states.

11.5 Root-performance ordering

Early time-to-output is sensitive to traversal order. For $n = 9$, root tasks are likely highly imbalanced. Recording root productivity and replaying roots in a learned order could improve early solution discovery while preserving completeness.

11.6 Hardware-counter-driven optimization

Every $n = 9$ optimization should be measured with hardware counters:

`cycles`, `instructions`, `cache-misses`, `LLC-load-misses`, `dTLB-load-misses`, `stalled-cycles`.

The main goal is to raise IPC above the observed sub-0.90 regime by making the memory access pattern sequential, local, and cache-aware.

12 Validation

Validation is separated from performance.

12.1 Mathematical validation

The program checks:

- the number of hypersimplex vertices $\binom{n}{3}$;
- input orbit counts for $n \leq 8$;
- uniform orbit volume and full vertex coverage;
- cell vertex masks satisfy all listed inequalities;
- pairwise face compatibility for accepted tilings;
- volume sum equals the full hypersimplex volume;
- final tiling coverage equals the full hypersimplex vertex set;
- canonical final keys are unique in quotient mode.

12.2 Small-case validation

The implementation has been repeatedly validated against:

$$n = 4 : \ 1, \qquad n = 5 : \ 3, \qquad n = 6 : \ 26$$

total quotient tilings including the trivial tiling. These are the production self-test counts. The self-test also verifies image generation and basic face tests for the small cases.

12.3 What is not validated by default

The implementation does not test realizability over a field, orientability, or regularity of a subdivision unless a specific optional module is added. It also does not claim that every generated tiling is regular. This distinction is essential: arbitrary matroid tilings and regular matroid subdivisions are different enumerative targets.

13 Visualization policy

DOT/Graphviz is used for process and dependency diagrams, and Python is used for numerical plots. Dense full graphs are deliberately avoided in the paper. When a graph is too dense to print legibly, the paper uses quotient diagrams, selected workflows, or simplified bottleneck maps, and full data are left to supplementary material.

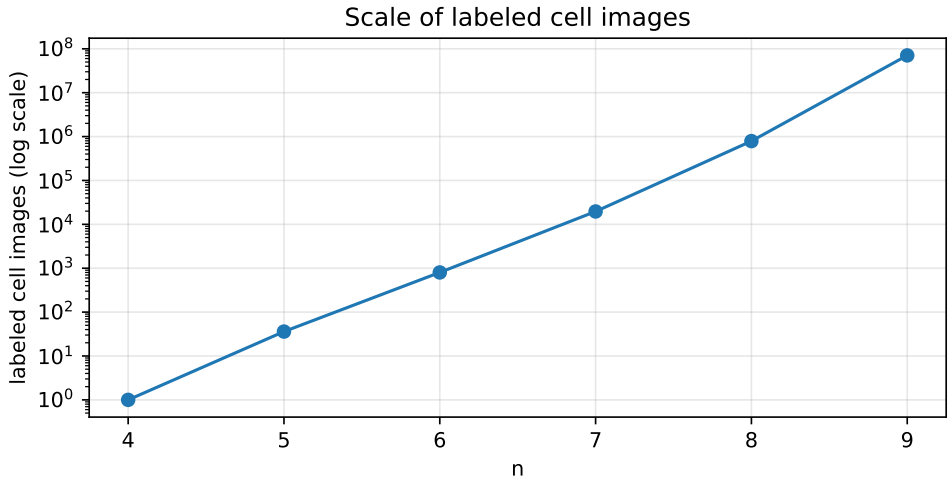

Figure 8: Growth in labeled cell images. The vertical axis is logarithmic. The jump from $n = 8$ to $n = 9$ changes the computational regime.

14 Reproducibility

The project is designed to make computations reproducible.

14.1 Build

A typical build is:

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release \
      -DALEXEEV_ENABLE_NATIVE=ON -DALEXEEV_ENABLE_LTO=ON
cmake --build build -j
```

For PGO:

```
cmake -S . -B build-pgo-gen -DCMAKE_BUILD_TYPE=Release \
      -DALEXEEV_ENABLE_NATIVE=ON -DALEXEEV_ENABLE_LTO=ON \
      -DALEXEEV_ENABLE_PGO_GENERATE=ON
cmake --build build-pgo-gen -j
./build-pgo-gen/alexeev_r3_cpp 8 --auto --gather-data \
      --gather-matrix quick --gather-solutions 1000
cmake -S . -B build-pgo-use -DCMAKE_BUILD_TYPE=Release \
      -DALEXEEV_ENABLE_NATIVE=ON -DALEXEEV_ENABLE_LTO=ON \
      -DALEXEEV_ENABLE_PGO_USE=ON
cmake --build build-pgo-use -j
```

14.2 Production commands

For a tuned $\Delta(3, 8)$ run:

```
./alexeev_r3_cpp 8 --data-dir data --cache-dir .alexeev_cache \
  --auto --auto-profile throughput --out r3n8_tilings.txt
```

For an initial $\Delta(3, 9)$ run:

```
./alexeev_r3_cpp 9 --data-dir data --cache-dir .alexeev_cache_n9 \
  --auto --auto-profile throughput --out r3n9_tilings.txt
```

14.3 Profiling workflow

Users on different systems should not inherit the Threadripper-tuned parameters blindly. The recommended workflow is:

```
./alexeev_r3_cpp 8 --auto --gather-data --gather-matrix core \
  --gather-cache-mode warm-profiled --gather-solutions 10000

python3 scripts/summarize_perf_data.py perf_data.txt
```

For $n = 9$, state-bounded runs are often more practical at first:

```
./alexeev_r3_cpp 9 --auto --gather-data --gather-matrix core \
  --gather-states 100000 --gather-solutions 0
```

The resulting `perf_data.txt` should be used to update `--auto` profiles for the local machine.

15 Limitations

The present implementation has several limitations.

1. The $n = 9$ path depends on a validated `allr3n9.txt` input file. It does not itself enumerate all rank–3 matroids from scratch.

2. The $n = 9$ partial-state cache is labeled, not quotient-canonical.
3. Regularity testing is not part of the production search.
4. Realizability and orientability are not tested.
5. Dense image-index bitsets become a major bottleneck for $n = 9$.
6. Current diagrams show reduced workflows, not full tiling adjacency graphs.
7. Benchmarks are machine-dependent and must not be interpreted as mathematical results.

16 Future work

16.1 For $\Delta(3, 8)$

The $n = 8$ engine is mature enough that future improvements should be narrow and benchmark-driven:

- PGO and LTO on the production machine;
- long-run benchmarks beyond the first 10,000 tilings;
- better root-performance ordering;
- final-solution external merge for very large output;
- more detailed lock-wait instrumentation.

16.2 For $\Delta(3, 9)$

The $n = 9$ engine requires a more substantial data-structure redesign:

- sparse-incompatible compatibility rows;
- compressed image-index bitsets;
- NUMA-local caches or process sharding;
- root-prefix task weighting;
- exact quotient partial-state canonicalization without brute $\mathfrak{S}_9$ scans;
- wide-mask kernels using `__uint128_t`, AVX2/AVX512 where measured useful, or two-limb hand-tuned kernels;
- hardware-counter-guided optimization to raise IPC.

16.3 Mathematical directions

Beyond raw enumeration, the implementation can support:

- databases of matroid tiling orbits;
- comparison of regular and nonregular tilings;
- computation of boundary strata in stable hyperplane arrangement compactifications;
- experiments with valuated matroids and Dressian cones;
- detection of realizable subfamilies if combined with oriented matroid or algebraic realizability tools.

17 AI use disclosure

Artificial intelligence tools were used during the development of this project to assist with code organization, debugging strategies, documentation drafts, mathematical exposition, performance-analysis summaries, and editorial revision. All mathematical definitions, algorithmic claims, implementation decisions, computational outputs, and final text were reviewed and validated by the author. The AI tools were not treated as authoritative sources for mathematical facts or computational results. AI assistance was used in developing the implementation and in drafting this paper.

18 Conclusion

The implementation described here makes Alexeev-style rank–3 matroid tiling computations substantially more concrete. The $\Delta(3,8)$ case has reached a strong benchmark-driven engineering state: the production path combines exact quotient search with efficient bit representations, compatibility caching, static-hybrid scheduling, and automatic profile tuning. The $\Delta(3,9)$ case is now accessible through a separate wide-mask engine, but its current performance profile reveals a new bottleneck regime dominated by memory hierarchy and data layout. The most important next step is therefore not another high-level search rewrite, but a careful redesign of compatibility-row and image-index representations for cache locality, NUMA locality, and sparse structure. If this succeeds, the computational program can push beyond the long-stale small cases and provide new experimental evidence for the combinatorics of stable weighted hyperplane arrangements.

References

[1] V. Alexeev, *Moduli of Weighted Hyperplane Arrangements*, Advanced Courses in Mathematics, CRM Barcelona, Birkhäuser/Springer, 2015.

[2] V. Alexeev, *Moduli of weighted hyperplane arrangements, with applications*, Barcelona lecture notes, 2013.

[3] J. Oxley, *Matroid Theory*, second edition, Oxford University Press, 2011.

[4] Y. Matsumoto, S. Moriyama, H. Imai, and D. Bremner, Matroid enumeration for incidence geometry, *Discrete & Computational Geometry* 47 (2012), 17–43.

[5] I. M. Gelfand, M. Goresky, R. MacPherson, and V. Serganova, Combinatorial geometries, convex polyhedra, and Schubert cells, *Advances in Mathematics* 63 (1987), 301–316.

[6] M. Develin and B. Sturmfels, Tropical convexity, *Documenta Mathematica* 9 (2004), 1–27.

[7] A. Fog, *Optimizing Software in C++: An optimization guide for Windows, Linux, and Mac platforms*, 2025 edition.

[8] B. Andrist and V. Sehr, *C++ High Performance: Boost and optimize the performance of your C++17 code*, Packt Publishing, 2018.

[9] S. Meyers, *Effective Modern C++*, O'Reilly Media, 2015.

[10] J. Ellson, E. Gansner, L. Koutsofios, S. North, and G. Woodhull, Graphviz and Dynagraph: static and dynamic graph drawing tools, in *Graph Drawing Software*, Springer, 2004.

A Appendix: representative benchmark tables

Table 2: Representative $\Delta(3, 8)$ time-to-10,000-tilings benchmark rows. These are computational measurements, not mathematical counts.

Profile	seconds	solutions/sec
cold static	101.0	99.0
warm static	87.5	114.3
hybrid 96	83.4	119.9
hybrid 144	66.5	150.5
hybrid 160	64.2	155.7
hybrid 160 + worker cache	62.9	158.9

Table 3: Representative $\Delta(3, 9)$ state-bounded root-static thread sweep.

threads	elapsed sec	states/sec	solutions/sec
48	104.73	95.49	12.53
96	90.18	110.89	14.07
128	85.27	117.59	14.17
144	89.62	111.99	13.59
160	90.75	110.36	13.04
176	94.87	105.55	12.15
192	109.95	91.27	10.54

B Appendix: supplied files

The paper package includes:

- `main.tex` and compiled `main.pdf`;
- DOT files and rendered PDF/PNG diagrams under `figures/`;

- Python plotting script under `scripts/`;
- CSV benchmark summaries under `data/`;
- small C++ code snippets under `code/`.